

Data Quality and Validation in ETL

Assignment — Completed Solutions

A full worked solution to the PW Skills assignment on data quality and validation in ETL pipelines, covering theory (data quality, duplicates, validation timing, business rules) and practical SQL work on the given Sales_Transactions and Customers_Master tables.

Theoretical Questions

Question 1: Define Data Quality in the context of ETL pipelines. Why is it more than just data cleaning?

Answer:

Data quality is the degree to which data is accurate, complete, consistent, unique, valid, and timely enough to be trusted for reporting and decision-making. In an ETL pipeline it is a broader, ongoing discipline rather than a single step: data cleaning only fixes surface-level issues such as nulls, typos, or formatting. Data quality goes further — it checks data against business rules (e.g. $\text{Txn_Amount} = \text{Quantity} \times \text{Price}$), enforces referential integrity between related tables, detects and resolves duplicates, and monitors consistency across source systems over time. Cleaning is a one-time fix; data quality is a continuous set of checks built into the pipeline.

Question 2: Explain why poor data quality leads to misleading dashboards and incorrect decisions.

Answer:

Dashboards simply aggregate whatever data flows into them, so any defect upstream is reproduced downstream. Duplicate transactions inflate revenue and quantity totals, missing values distort averages and completion rates, and orphaned or mismatched IDs cause records to disappear from joined reports entirely. Because dashboards present numbers with an appearance of authority, decision-makers tend to trust them without question — so an inflated sales figure or an undercounted churn rate can directly lead to wrong inventory planning, incorrect targets, or misallocated budget, all traced back to a data quality issue no one caught early.

Question 3: What is duplicate data? Explain three causes in ETL pipelines.

Answer:

Duplicate data refers to multiple records in a dataset that represent the same real-world event or entity, when there should logically be only one. Common causes in ETL pipelines include:

- **Re-ingestion of source files** — rerunning a failed or scheduled ETL job without a dedup/idempotency check reloads records that were already ingested.
- **Multi-system integration** — the same customer or transaction exists in more than one source system (e.g. CRM and POS) and gets merged into the warehouse without proper key matching.
- **User or application retries** — a double form submission, a page refresh after a slow response, or a retried API call after a timeout creates a second identical transaction record.

Question 4: Differentiate between exact, partial, and fuzzy duplicates.

Answer:

- **Exact duplicates** — records that are identical across every field. Example: Txn_ID 201 and 208 in the dataset both have Customer_ID C101, Product_ID P11, Quantity 2, Amount 4000, Date 2025-12-01 and City Mumbai — a perfect match apart from the Txn_ID itself.

- **Partial duplicates** — records that match on the core business key but differ in some secondary or system-generated fields (timestamp, transaction ID, metadata). They represent the same underlying event but were logged with slightly different auxiliary details.
- **Fuzzy duplicates** — records that are not byte-for-byte identical but represent the same real-world entity due to typos, formatting differences, or variant spellings (e.g. 'Rahul Mehta' vs 'Rahul Mehtaa', or a phone number written with and without a country code). These require similarity/fuzzy-matching logic rather than a simple equality check to detect.

Question 5: Why should data validation be performed during transformation rather than after loading?

Answer:

Validating during transformation catches problems while the data is still isolated in a staging area, before it reaches the warehouse or any report that downstream users might already be consuming. This is cheaper and safer: fixing or rejecting a bad record in staging costs far less than issuing a correction after dashboards, models, or business decisions have already been built on it. It also lets the pipeline enforce data contracts early — invalid records can be quarantined or flagged before loading — keeping the load stage a trustworthy, 'clean by construction' zone rather than something that constantly needs retroactive fixes.

Question 6: Explain how business rules help in validating data accuracy. Give an example.

Answer:

Business rules encode domain knowledge as explicit conditions that data must satisfy to be considered correct, going beyond generic checks like 'not null'. They catch errors that are technically well-formed but logically wrong. For example, a rule such as **$\text{Txn_Amount} = \text{Quantity} \times \text{Unit_Price}$** would flag any transaction where the total doesn't match the expected calculation, and a rule such as **Quantity must be greater than 0** would immediately flag Txn_ID 205 in the dataset, where Quantity is Null — a row that passes a basic 'field exists' check but fails the business rule that every sale must have a positive quantity.

Practical Questions

Sales_Transactions dataset (as given):

Txn_ID	Customer_ID	Customer_Name	Product_ID	Quantity	Txn_Amount	Txn_Date	City
201	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai
202	C102	Anjali Rao	P12	1	1500	2025-12-01	Bengaluru
203	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai
204	C103	Suresh Iyer	P13	3	6000	2025-12-02	Chennai
205	C104	Neha Singh	P14	Null	2500	2025-12-02	Delhi
206	C105	N/A	P15	1	Null	2025-12-03	Pune
207	C106	Amit Verma	P16	1	1800	Null	Pune
208	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai

Customers_Master dataset (as given):

CustomerID	CustomerName	City
C101	Rahul Mehta	Mumbai
C102	Anjali Rao	Bengaluru
C103	Suresh Iyer	Chennai
C104	Neha Singh	Delhi

Question 7: SQL query to list all duplicate keys and their counts using the business key (Customer_ID + Product_ID + Txn_Date + Txn_Amount)

```
SELECT
Customer_ID,
Product_ID,
Txn_Date,
Txn_Amount,
COUNT(*) AS duplicate_count
FROM Sales_Transactions
GROUP BY Customer_ID, Product_ID, Txn_Date, Txn_Amount
HAVING COUNT(*) > 1;
```

Result on the given dataset:

Customer_ID	Product_ID	Txn_Date	Txn_Amount	duplicate_count
C101	P11	2025-12-01	4000	3

Txn_ID 201, 203 and 208 all share the same business key (C101, P11, 2025-12-01, 4000), so this key appears **3 times** — 2 of those are duplicate occurrences beyond the first legitimate record.

Question 8: Enforcing Referential Integrity — detect Sales_Transactions.Customer_ID values that do not exist in Customers_Master

Identifying the violations:

Comparing every Customer_ID in Sales_Transactions against Customers_Master, **C105** (Txn_ID 206) and **C106** (Txn_ID 207) do not appear in Customers_Master at all — these are referential integrity violations (orphan foreign keys). All other Customer_ID values (C101, C102, C103, C104) exist correctly in the master table.

SQL query to detect such violations:

```
SELECT DISTINCT st.Txn_ID, st.Customer_ID
FROM Sales_Transactions st
LEFT JOIN Customers_Master cm
ON st.Customer_ID = cm.CustomerID
WHERE cm.CustomerID IS NULL;
```

Result on the given dataset:

Txn_ID	Customer_ID	Issue
206	C105	No matching record in Customers_Master
207	C106	No matching record in Customers_Master

Note: this dataset is a synthetic training example built purely for the assignment; no real customer data is involved.